La méthode des différences finies

One-dimensional steady diffusion problem and sparse matrix

We are interested to solve the diffusion problem with Dirichlet Boudary condition

$$-vu''(x) = f(x), x \in [0, L]$$

$$u(0) = \alpha, u(L) = \beta$$

We consider the case where $v = 1, L = 1, \alpha = \beta = 1$ and $f(x) = 4\frac{\pi^2}{L}cos(\frac{2\pi}{L}x)$ We discretized the interval $I = [0, L]$

```
u = 1;
L = 1;
alpha = 1;
beta = 1;
f = @(x) 4 * pi^2 * cos(x);
```

First of all, we have to find an exact solution:

$$y_{exact} = \frac{4\pi^2}{u}cos(x) + x\frac{(\beta - \alpha + \frac{4\pi^2}{u}(1 - cos(L))}{L} + (\alpha - \frac{4\pi^2}{u})$$

Then we have to initialize the constants like $a = 0, b = 1, N, h = (b - a)/N$, and we have to creat some blanc vectors in order to store the data in them.

```
exacteSol = @(x) 4*pi^2*cos(x)/u + x*(beta-alpha + 4*pi^2/u*(1-cos(L)))/L + (alpha - 4*pi^2/u) ;
```

Here comes to the implement of the matrix which corresponds to the finite scheme to solve the problem.

```
F = h.^2*f(x') + u *[alpha; zeros(N-1, 1); beta];
K = u*(diag(-ones(1, N),1) + diag(2*ones(1, N+1)) + diag(-ones(1, N),-1));
```

Using Matlab matrix inversion, we compute uh

> https://www.mathworks.com/help/matlab/ref/inv.html

Y = inv(X) computes the inverse of square matrix X.

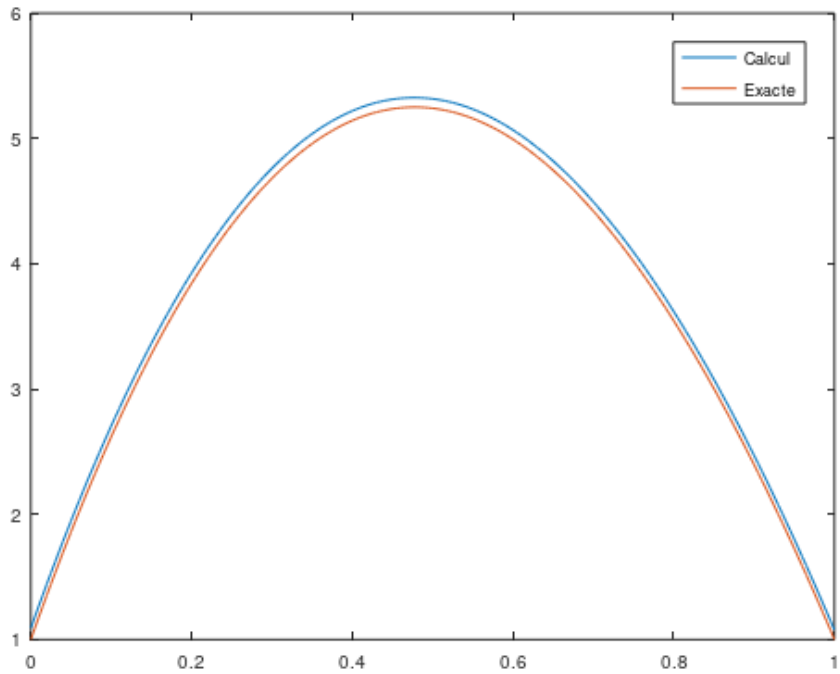
- X^(-1) is equivalent to inv(X).
- x = A\b is computed differently than x = inv(A)*b and is recommended for solving systems of linear equations.

$$Au = F$$

```
uh=inv(K)*F
%or
uh = F/K
```

Then we plot the found solution uh against the exact solution

```
u_cal = calculInverse(K, F, 'jacobi');

u_exa = exacteSol(x)';

plot(x, u_cal);
hold on;
plot(x, u_exa);
legend('Calcul', 'Exacte');
hold off;
```



Now we study the convergence

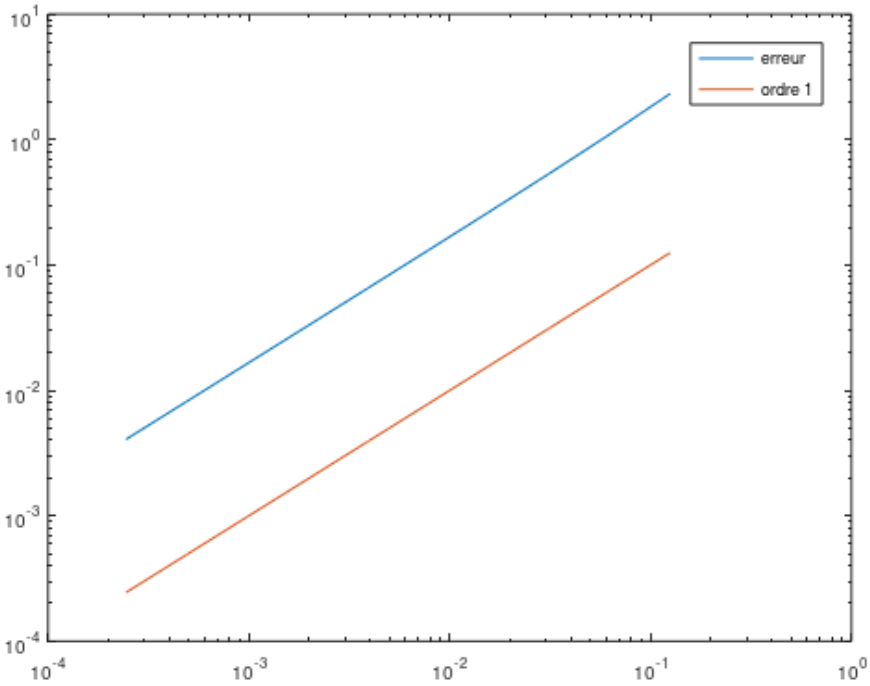
$$err = \sqrt{\frac{\sum_{j=1}^{N+1}(u_j - u_{ex,j})^2}{N+1}}$$

We need *Jacobi* and *Gauss-seidel* algorithms to solve the problem $Ax = b$

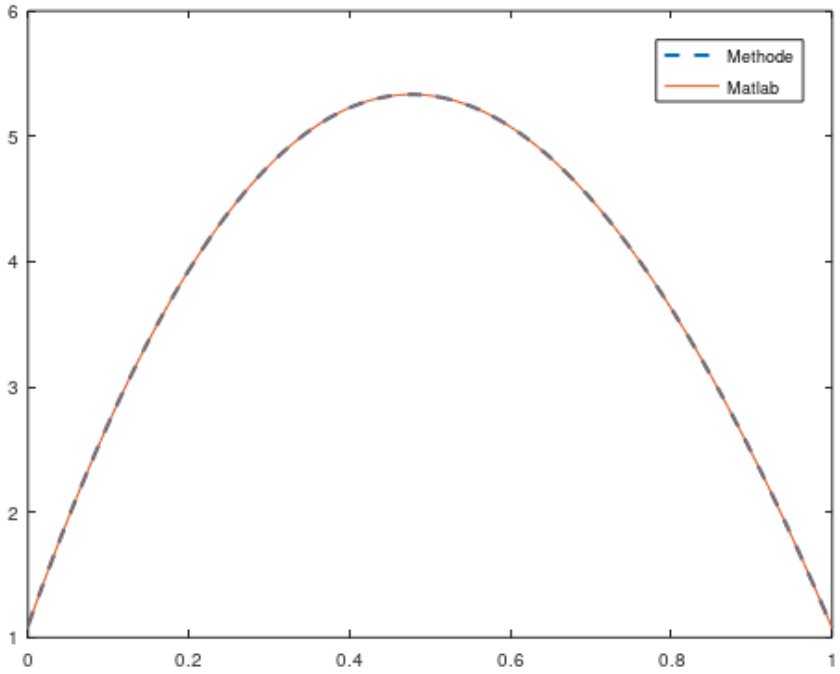
Further more, we can simplify the algorithm with *sparse* (which conserve the necessary data)

```
err = zeros(1, nTest);
hVec = zeros(1, nTest);

for i = 1:nTest
    N = 4*2^i;

    h = (L-l)/N;
    x = l:h:L;

    F = h.^2*f(x') + u *[alpha; zeros(N-1, 1); beta];
    K = u*(diag(-ones(1, N),1) + diag(2*ones(1, N+1)) + diag(-ones(1, N),-1));

    u_cal = calculInverse(K, F, 'matlab');
    u_exa = exacteSol(x)';

    err(i) = norm(u_cal - u_exa)/sqrt(N+1);
    hVec(i) = h;
end
```

```
loglog(hVec, err);
hold on;
loglog(hVec, hVec.^1);
legend('erreur', 'ordre 1');
hold off;
```




```
N = 200;
l = 0;
h = (L-l)/N;
x = l:h:L;

F = h.^2*f(x') + u *[alpha; zeros(N-1, 1); beta];
K = u*(diag(-ones(1, N),1) + diag(2*ones(1, N+1)) + diag(-ones(1, N),-1));

u_jac = calculInverse(K, F, '-.', 'g-s');

u_mat = calculInverse(K, F, 'matlab');

plot(x, u_jac, '--', 'LineWidth', 2);
hold on;
plot(x, u_mat);
legend('Methode', 'Matlab');
hold off;
```



Output the computational time spent in each function

```
N = 400;
h = L/N;
x = 0:h:L;

F = h.^2*f(x') + u *[alpha; zeros(N-1, 1); beta];
K = u*(diag(-ones(1, N),1) + diag(2*ones(1, N+1)) + diag(-ones(1, N),-1));
disp('-----------VITESSE DE CONVERGENCE DU CALCUL D INVERSE-----------');
disp('JACOBI : ');
tic
calculInverse(K, F, 'jacobi');
toc
disp('JACOBI 2 : ');
tic
calculInverse(K, F, 'jacobi2');
toc
disp('MATLAB : ');
tic
calculInverse(K, F, 'matlab');
toc
disp('GAUSS-SEIDEL : ');
tic
calculInverse(K, F, 'g-s');
toc
```

- ----------VITESSE DE CONVERGENCE DU CALCUL D INVERSE----------
  JACOBI :
  Elapsed time is 10.7289 seconds.
  JACOBI 2 :
  Elapsed time is 17.6156 seconds.

MATLAB :

Elapsed time is 0.015388 seconds.

GAUSS-SEIDEL :

Elapsed time is 5.40548 seconds.

the implement method :

```
function [ x ] = calculInverse( A, b, method)
%   Summary of this function goes here
%   Detailed explanation goes here
    tol = 10^(-5);
    k = 0; % compte le nombre d'iteration
    err = 1;
    max = 200000;
    x = zeros(length(A), 1);

    if strcmp(method, 'jacobi')
        D = diag(diag(A));
        E = - tril(A) + D;
        F = - triu(A) + D;

        M_1 = sparse(diag(1./diag(D)));
        N = sparse(E+F);
        while (err > tol && k<max)
            x_prec = x;
            x = M_1 * (N * x_prec + b);
            err = norm(x_prec - x);
            k = k + 1;
        end
        if (k == max)
            disp('ARRET NOMBRE ITERATIONS');
        end

    elseif strcmp(method, 'jacobi2')
        d = diag(A);
        n = length(x);

        while (err > tol && k < max)
            x_prec = x;
            x = ([x(2); x(1:(n-2)) + x(3:n) ; x(n-1)] + b)./d;
            err = norm(x_prec - x);
            k = k + 1;
        end
        if (k == max)
            disp('ARRET NOMBRE ITERATIONS');
        end

    elseif strcmp(method, 'g-s')
        D = diag(diag(A));
        L = tril(A) - D;
        U = triu(A) - D;

        M = sparse(L+D);
        U = sparse(U);
        while (err > tol && k < max)
            x_prec = x;
            x = M\(b-U*x);
            err = norm(x_prec - x);
            k = k + 1;
```

```
        end
        if (k == max)
            disp('ARRET NOMBRE ITERATIONS');
        end

    else
        % matlab method
        x = (A\b);
    end

end
```